

Natural Computing SoSe25

Exercise Sheet 5 — July 3rd, 2025

Thomas Gabor, Maximilian Zorn

1 Optimization 101

(i) To deepen your understanding of the optimization concepts introduced in the lecture, define $\mathcal{X}$, $\mathcal{T}$, and τ , as well as an example for $x \in \mathcal{X}$, for the following example problems. Also state if τ is to be *maximized* or *minimized*.

Recall the following example based on the lecture:

Knapsack Problem: *Maximize the combined value of stored items $o \in \{A, \dots, E\}$ with some $\text{value}(o)$ and some $\text{weight}(o)$ within some given total weight capacity M .*

$$\mathcal{X} = \{x \mid x \subseteq \{A, \dots, E\} \wedge \sum_{o \in x} \text{weight}(o) \leq M\}$$

$$x =_{(\text{e.g.})} \{A, B, E\}$$

$$\mathcal{T} = \mathbb{N}$$

$$\text{max. } \tau = x \mapsto \sum_{o \in x} \text{value}(o)$$

(a) Traveling Salesperson Problem (TSP): *Given V , which is a set of $|V| = n$ destinations, find the fastest route for the salesperson so that the distance they have to travel between all destinations (e.g., $D(i, j)$ between destinations $i, j \in V$) is minimal. They have to visit every destination exactly once and need to end up in the destination they started at.*

$$\mathcal{X} =$$

$$x =_{(\text{e.g.})}$$

$$\mathcal{T} =$$

$$\text{min./max. } \tau =$$

(b) Longest common beginning substring problem: For a set of words S , e.g., $S = \{ \text{"AACBAB"}, \text{"CABBAA"} \}$, with $S \subset \{ \text{"A"}, \text{"B"}, \text{"C"} \}^* = \{ \langle c_i \rangle_{1 \leq i \leq N} \mid c_i \in \{ \text{"A"}, \text{"B"}, \text{"C"} \} \wedge N \in \mathbb{N} \}$ find the longest string of characters that appears at the start of all words of S .

$\mathcal{X} =$

$x =_{(e.g.)}$

$\mathcal{T} =$

min./max. $\tau =$

(c) (Bonus:) Longest common ~~beginning~~ substring problem: For a set of words S as given above, find the longest string of characters that appears somewhere within all words of S .

$\mathcal{X} =$

$x =_{(e.g.)}$

$\mathcal{T} =$

min./max. $\tau =$

(d) (Bonus:) Chess: We search for a strategy for a black player in a game of chess against a deterministic white player where the player tries to win by minimizing the total value of the opponent's chess pieces (by taking them). A strategy is (in this case) a series of valid chess moves. Let $\mathcal{M}$ be the set of all possible (valid or invalid) chess moves. When B is the set of all board configurations and $b_0 \in B$ is the starting configuration, let $M(b)$ for $b \in B$ be the set of all valid moves for black given the board b . For $\mu \in M(b)$, let $\mu(b) \in B$ be the resulting board configuration when playing the move μ . Let $w(b) \in B$ be the deterministic resulting board configuration when white makes a move on board b . Let $P = \text{white_pieces}(b)$ be a series of all remaining white pieces in a fixed order, i.e., $P = \langle P_j \rangle_{1 \leq j \leq |P|}$ with $P_j \in \{ \text{"Pawn"}, \dots, \text{"King"} \}$ for all j . Pawns are considered worth one point, knights and bishops about three points each, rooks about five points (the value difference between a rook and a bishop or knight being known as the exchange), and queens about nine points. The king is more valuable than all of the other pieces combined, since its checkmate loses the game.

$\mathcal{X} =$

$x =_{(e.g.)}$

$\mathcal{T} =$

min./max. $\tau =$

(ii) In the lecture we have also introduced the notions of *working to some extent* and *improving*. For each of the following minimization scenarios, draw a 1D solution landscape of your choice, where ...

(a) an optimization run with the *stochastic hill climbing* algorithm (with a neighborhood of $x \pm N$) is (almost) certain to be improving after g steps and where the *end result* is always the *best result*.

(b) an optimization run with the *stochastic hill climbing* algorithm (with a neighborhood of $x \pm N$) is (almost) certain to be improving after g steps and where the *end result* is (almost) never the *best result*.

(c) an optimization run with the *gradient descent* algorithm is (almost) certain to be

working to some extent after g steps and where the *end result* is (almost) never the *best result*.

(iii) What are the main weaknesses of the *stochastic hill climbing* algorithm and the *gradient descent* algorithm?

2 Bright Side Search

We give a shortened version of the definition for optimization processes from the lecture and then introduce a new optimization algorithm called *bright side search*.

Definition 1 (optimization (shortened)). *Let $\mathcal{X}$ be an arbitrary state space. Let $\mathcal{T}$ be an arbitrary set called target space and let $\leq$ be a total order on $\mathcal{T}$. A total function $\tau : \mathcal{X} \rightarrow \mathcal{T}$ is called target function. Optimization (minimization/maximization) is the procedure of searching for an $x \in \mathcal{X}$ so that $\tau(x)$ is optimal (minimal/maximal). Unless stated otherwise, we assume minimization.*

An optimization run of length $g+1$ is a sequence of states $\langle x_t \rangle_{0 \leq t \leq g}$ with $x_t \in \mathcal{X}$ for all t .

Let $e : \langle \mathcal{X} \rangle \times (\mathcal{X} \rightarrow \mathcal{T}) \rightarrow \mathcal{X}$ be a possibly randomized or non-deterministic function so that the optimization run $\langle x_t \rangle_{0 \leq t \leq g}$ is produced by calling e repeatedly, i.e., $x_{t+1} = e(\langle x_u \rangle_{0 \leq u \leq t}, \tau)$ for all t , $1 \leq t \leq g$, where x_0 is given externally (e.g., $x_0 =_{\text{def}} 42$) or chosen randomly (e.g., $x_0 \sim \mathcal{X}$). An optimization process is a tuple $(\mathcal{X}, \mathcal{T}, \tau, e, \langle x_t \rangle_{0 \leq t \leq g})$.

Algorithm 1 (bright side search). *Let $\mathcal{D} = (\mathcal{X}, \mathcal{T}, \tau, e, \langle x_u \rangle_{0 \leq u \leq t})$ be an optimization process. We assume $\mathcal{X} = [0; 100] \subset \mathbb{N}$ and $\mathcal{T} = [0; 100] \subset \mathbb{N}$. The process $\mathcal{D}$ continues via bright side search iff e is given via*

$$e(\langle x_u \rangle_{0 \leq u \leq t}, \tau) = x_{t+1} = \begin{cases} \lceil x_t/2 \rceil & \text{if } \tau(x_t - 1) < \tau(x_t + 1), \\ \lfloor x_t + (100 - x_t)/2 \rfloor & \text{otherwise.} \end{cases}$$

For $x \notin [0; 100]$ we assume that $\tau(x) = \infty$.

Note that $\lceil x \rceil$ is x rounded up to the nearest integer and $\lfloor x \rfloor$ is x rounded down to the nearest integer.

(a) Let the target function to be minimized be

$$\tau(x) = |x - 42|.$$

Let the initial state be $x_0 = 65$. **Execute bright side search (cf. Algorithm 1) in this setting for three time steps, i.e., compute x_1, x_2, x_3 . Briefly state why this run will never reach the global optimum at $x^* = 42$.**

(b) We say an optimization algorithm converges on the global optimum if (given enough time) it is likely to come closer to the global optimum than it was initially and if, once it reaches the global optimum, it stays close to that point.

Give a target function $\tau^\dagger : \mathcal{X} \rightarrow \mathcal{T}$ in accordance with Algorithm 1 with one single global optimum so that bright side search always reaches the global optimum. Give that global optimum and briefly state why bright side search can reach it. Then show that bright side search still does not converge on that global optimum.